

Report - Version 7: GRU Language Model

Introduction

Version 7 extends the TensorFlow recurrent neural character language model from Version 6 with a gated recurrent unit (GRU).

The corpus, vocabulary, four-character context windows, learned embeddings, 16-dimensional hidden state, training and validation split, shuffled mini-batches, cross-entropy objective, automatic differentiation, and autoregressive sampling remain in place. Instead of updating the hidden state with one tanh transformation, the model now learns update and reset gates together with a candidate hidden state.

```
Characters
↓
Integer identifiers
↓
4-character context
↓
Trainable embeddings
↓
GRU gated hidden states
↓
Output weights
↓
Generated text
```

Goal of Version 7

The model should be able to:

- reuse the same 440-character corpus, 27-character vocabulary, and four-character context windows from Version 6;
- reuse trainable 8-dimensional character embeddings and a 16-dimensional recurrent hidden state;
- replace the simple recurrent update with an explicit gated recurrent unit;
- learn update-gate, reset-gate, candidate-state, embedding, and output weights with `tf.GradientTape`;
- preserve mini-batch training, validation monitoring, and reproducible autoregressive generation;
- verify GRU shapes, gate ranges, gradients, parameter updates, deterministic retraining, and generation.

Architecture

The complete Version 7 flow is:

```
Corpus: 440 characters
↓
Vocabulary: 27 characters
↓
436 four-character context-target examples
↓
Integer input tensor: (436, 4)
↓
Embedding matrix: (27, 8)
↓
Sequential GRU processing
↓
Update gate + reset gate + candidate state
↓
Final hidden state: 16 values
↓
Output weight matrix: (16, 27)
↓
Reproducible 80 / 20 example split
↓
GradientTape mini-batch updates
↓
Sliding-window autoregressive generator
```

The context still contains four previous characters. Version 7 processes their embeddings from left to right and reuses the same GRU weights at every position. The update gate controls how much of the previous hidden state is preserved, while the reset gate controls how previous information contributes to the candidate state.

Context windows, split and mini-batches

Sliding windows create the same 436 input-target examples as Version 6. Each example contains four ordered character identifiers. The hidden state starts from zeros for every context example and is updated once for each character by the GRU.

```
training examples = 348
validation examples = 88
batch size = 32
updates per epoch = 11
```

Only training examples are shuffled and used for parameter updates. Validation examples measure the loss without modifying the model. Neighboring context windows still overlap, so the example-level random split retains the same intentional small-dataset limitation as Version 6.

Trainable embeddings and gated recurrent weights

Each character identifier selects one row from the trainable embedding matrix. The four selected vectors are processed sequentially. At every position, the current embedding and previous hidden state are used to compute an update gate, a reset gate, and a candidate hidden state.

```
Input tensor shape: (436, 4)
Embedding matrix shape: (27, 8)
Embedded context shape: (1, 4, 8)
Update input weights shape: (8, 16)
Update recurrent weights shape: (16, 16)
Reset input weights shape: (8, 16)
Reset recurrent weights shape: (16, 16)
Candidate input weights shape: (8, 16)
Candidate recurrent weights shape: (16, 16)
Final hidden state shape: (1, 16)
Output weight matrix shape: (16, 27)
Trainable parameters: 1800
```

The embedding matrix contains 216 values. Each of the three GRU transformations contains 128 input weights and 256 recurrent weights, and the output matrix contains 432 values. The model therefore has 1800 trainable parameters, compared with 1032 in Version 6.

```
z_t = sigmoid(x_t W_z + h_(t-1) U_z)
r_t = sigmoid(x_t W_r + h_(t-1) U_r)
candidate_t = tanh(x_t W_h + (r_t * h_(t-1)) U_h)
h_t = z_t * h_(t-1) + (1 - z_t) * candidate_t
```

Stable TensorFlow loss

The context identifiers are mapped to embeddings and processed by the GRU. The final hidden state is multiplied by the output weights to produce next-character logits. Sparse softmax cross-entropy measures the prediction error directly from the logits and integer target IDs.

```
_, logits = gru_forward(..., inputs)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

The initial train and validation losses are both about 3.2958, consistent with an almost uniform prediction over the 27-character vocabulary.

Controlled execution and reproducibility

The notebook keeps CPU execution and suppresses low-level TensorFlow logs. Explicit seeds control TensorFlow parameter initialization, NumPy data splitting, mini-batch shuffling, and text sampling. Repeated runs are deterministic within the same software environment.

Automatic differentiation

For each mini-batch, `tf.GradientTape` records the gated recurrent forward calculations and derives the loss with respect to all eight trainable parameter matrices:

- `tf.GradientTape` records embedding lookup, update and reset gates, candidate states, hidden-state updates, output projection, and loss;
- `tape.gradient` calculates gradients for the embedding, six GRU weight matrices, and output weights;
- the sparse embedding gradient is converted to a dense tensor for this small vocabulary;
- `assign_sub` applies gradient descent updates to all eight variables;
- training and validation loss are recorded from the current parameters at each epoch.

```
with tf.GradientTape() as tape:  
    batch_loss = calculate_loss(...)  
    gradients = tape.gradient(batch_loss, trainable_variables)
```

Training configuration:

```
Learning rate: 1.0  
Batch size: 32  
Epochs: 100  
Seed: 42
```

Training result

```
Initial train loss: 3.295838  
Final train loss: 2.659556  
Initial validation loss: 3.295836  
Final validation loss: 2.913941  
Best validation loss: 2.899463  
Best validation epoch: 99
```

The GRU learns slowly during the first part of training. The losses remain almost unchanged through roughly 50 epochs, then decrease more clearly after about epoch 60. Training loss continues to improve through epoch 100, while validation loss reaches its minimum at epoch 99.

Validation loss increases slightly at the final epoch after reaching its minimum. This suggests the beginning of mild overfitting. The notebook records the best epoch for analysis but intentionally keeps the final epoch-100 parameters for generation.

Gradient, gate and parameter checks

The tests verify finite gradients for all eight trainable parameter matrices, matching gradient shapes, finite hidden states, valid gate ranges, and that every parameter matrix actually changes during training.

```
0 <= update_gate <= 1
0 <= reset_gate <= 1
-1 <= candidate_state <= 1
-1 <= hidden_state <= 1
assert all(gradient is not None for gradient in gradients)
assert np.max(abs(initial - trained)) > 0.0
```

Learned probabilities

After training, the model produces a next-character probability distribution from an ordered four-character context. For the context mode, the highest probabilities are:

```
space: 0.2329 e: 0.0918 s: 0.0825
t: 0.0765 n: 0.0706 i: 0.0636
```

Neural text generation

For every prediction, the current four-character window is mapped to embeddings and processed from left to right by the GRU. The hidden state starts from zeros for that window, the final state produces the logits, and a local NumPy generator samples the next character. The oldest context character is then removed and the window advances.

```
mode -> sampled character
ode? -> next sampled character
...
```

Five samples from the trained context mode with seed 42 are r, e, s, o, and space. The demonstration then generates 300 characters from the final parameters, and the same seed reproduces the same text.

Included checks

The notebook assertions verify context-window construction, tensor and variable types, all GRU parameter shapes, gate and hidden-state ranges, data splits, finite losses, normalized probabilities, finite gradients, parameter changes, deterministic retraining, reproducible generation, requested output length, and the starting context.

A complete top-to-bottom run ends with the message `All checks passed.`

What Version 7 teaches

The central progression is:

context -> IDs -> embeddings -> GRU gates -> hidden state -> logits
-> loss -> gradients -> update -> slide -> sample

Version 7 introduces gated recurrence. The educational focus moves from the single tanh update of Version 6 to learned mechanisms that control how previous information is preserved and how candidate information enters the hidden state.

Interpretation

The four context embeddings are still processed sequentially. The update gate interpolates between the previous hidden state and the candidate state, while the reset gate controls how previous information contributes to the candidate computation. The final 16-dimensional hidden state summarizes the ordered context before the output projection.

The model has 1800 parameters because the GRU learns three recurrent transformations instead of one. The best validation loss is 2.8995 at epoch 99. This version is designed to expose gated recurrence, not to claim that more parameters must improve validation performance on this small corpus.

Limitations

- the context still has a fixed length of four characters;
- the hidden state starts from zeros for every training example;
- generation recomputes each four-character window from a zero hidden state instead of preserving state indefinitely;
- the corpus is small and embedded directly in the notebook;
- overlapping context windows can appear across the random training and validation split;
- the GRU intentionally uses no bias terms;
- training uses plain mini-batch gradient descent with a fixed learning rate;
- there is no early stopping or checkpoint restoration, so generation uses epoch-100 parameters even though validation is best at epoch 99;
- generated text captures local sequential patterns but still has no long-term coherence.

Conclusion

Version 7 replaces the simple recurrent update of Version 6 with an explicit GRU. Character IDs select trainable embeddings, the GRU processes them from left to right through update and reset gates, and the final hidden state predicts the next character.

The implementation remains inspectable while introducing learned control over recurrent information flow. It completes the gated recurrent stage and prepares the next version for attention mechanisms.